

Journal 03: Understanding Multithreaded Systems and Performance

Sharie Rhea

Southern New Hampshire University

CS 510: Operating Systems Principles and Applications

Brian Enochson

February 22, 2026

Today's computers can run multiple concurrent processes through context switching, and on multicore systems, truly in parallel. Beyond the process lies an even smaller unit of work for the central processing unit (CPU): the thread. Multithreading can dramatically improve performance and better utilize available hardware resources. This essay first outlines the core components of a thread, then clarifies the relationship between threads and processes. It next examines multithreading and performance optimization, and finally compares thread management in Microsoft Windows, macOS, and Linux.

A thread consists of several components stored in the thread control block (TCB). The thread ID uniquely identifies the thread so the operating system can manage it. To maintain execution state, each thread has its own program counter (PC), which points to the next instruction, and a stack pointer (SP), for accessing memory (Silberschatz et al., 2023). Although threads within the same process share most data, each thread maintains a private stack accessed through its own SP (Silberschatz et al., 2023). The TCB also records the thread's state—such as runnable or blocked—and includes a pointer to the parent process (Silberschatz et al., 2023).

With this structure in mind, it becomes clearer how threads relate to processes. A process contains one or more threads. When a process contains multiple threads, it is considered multithreaded. Threads belonging to the same process share the code and data sections, as well as operating system resources such as file descriptors (Silberschatz et al., 2023). However, each thread retains its own PC and SP. Because threads share memory, careful synchronization is required. Critical sections are portions of code that must not execute concurrently—for example, when they modify the same data (Silberschatz et al., 2023). To ensure correctness, four conditions must be considered. First, mutual exclusion ensures that only one thread enters the

critical section at a time. Second, progress must be guaranteed so that waiting threads do not lock out others from entering. Third, one thread must not repeatedly enter the critical section and starve others. Finally, if two threads attempt to enter at the same time, they must not indefinitely block each other and create a deadlock (Silberschatz et al., 2023).

Building on these synchronization concerns, multithreading more broadly refers to dividing a process into multiple concurrent or parallel threads, each performing distinct work. Although thread creation is less expensive than process creation, it still incurs overhead, including metadata storage and scheduling costs. Thread pools mitigate this overhead by pre-allocating a fixed number of worker threads and assigning tasks as threads become available (Silberschatz et al., 2023). If no threads are free, tasks wait in a queue.

Threading lifecycle management is another challenge. Threads may transition among new, runnable or running, blocked, and terminated states (Silberschatz et al., 2023). Early termination must also be handled carefully. In applications such as search, once one thread finds a result, others may become unnecessary. Deferred cancellation allows a thread to periodically check whether it should terminate voluntarily, whereas asynchronous cancellation forcibly stops a thread via a signal (Silberschatz et al., 2023). Because asynchronous cancellation can leave shared data in inconsistent states, deferred cancellation is generally preferred.

The primary motivation for multithreading is performance. However, its benefits are bounded by Amdahl's Law, which states that overall speedup is limited by the portion of a program that must execute serially (Silberschatz et al., 2023). As the serial fraction increases, the maximum possible speedup decreases; for example, a program that is 50 percent serial cannot achieve more than a two times speedup. Parallelization typically takes one of two forms. Data

parallelism divides a dataset among multiple threads performing the same operations. One example of this would be summing separate halves of an array before combining the results (Silberschatz et al., 2023). Task parallelism, by contrast, assigns different operations to different threads, which may or may not access the same data. For example, computing an average while simultaneously searching a database.

Finally, operating systems differ in how they implement threading models. Threads may exist at the user or kernel level, and user-level threads must be mapped to kernel threads through models such as many-to-one, one-to-one, or many-to-many (Silberschatz et al., 2023). Linux uses the POSIX Pthreads library and follows a one-to-one model (Silberschatz et al., 2023). Windows also employs a one-to-one model with its own threading application programmer interface (API). macOS relies on Apple's Grand Central Dispatch (GCD), which manages tasks through prioritized dispatch queues layered over kernel threads (Silberschatz et al., 2023).

In summary, multithreading enables modern systems to use hardware resources efficiently by executing tasks concurrently or in parallel. At the same time, it introduces synchronization, lifecycle, and performance constraints that must be carefully managed. Different operating systems address these challenges through distinct threading models, but all aim to balance efficiency, responsiveness, and correctness.

References

Silberschatz, A., Galvin, P. B., & Gagne, G. (2023). *Operating System Concepts* (10th ed.).

Wiley. <https://learn.zybooks.com/zybook/CS-510-10435.202617-1>